

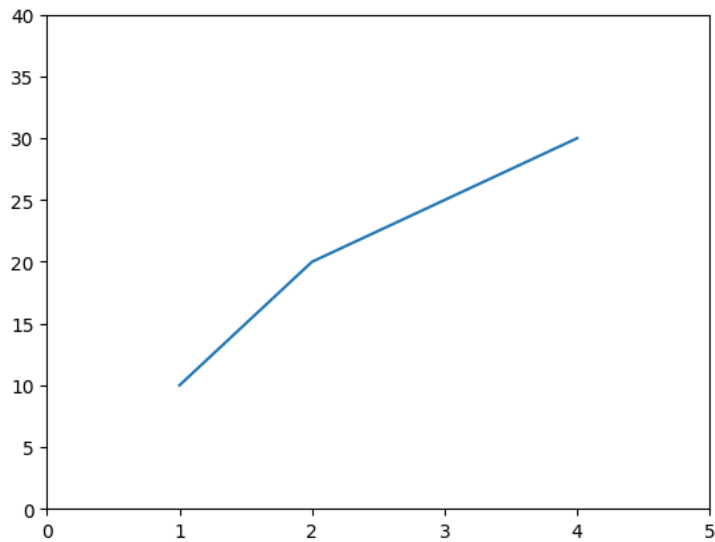
Controlling axes is a key part of making your data visualizations clear, accurate, and meaningful. In tools like Matplotlib, axes control helps you adjust how data is displayed on the X and Y axes.

```
import matplotlib.pyplot as plt

plt.plot([1,2,3,4], [10,20,25,30])

plt.xlim(0, 5)
plt.ylim(0, 40)

plt.show()
```

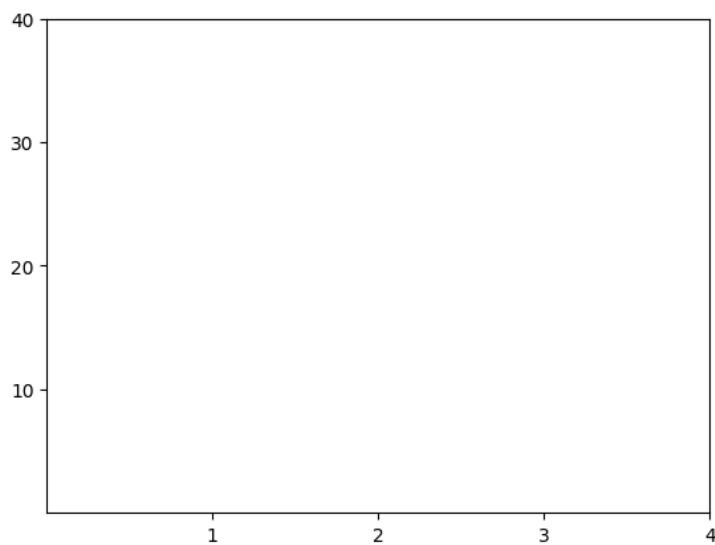


Setting Axis Ticks

Ticks are the marks/numbers on axes.

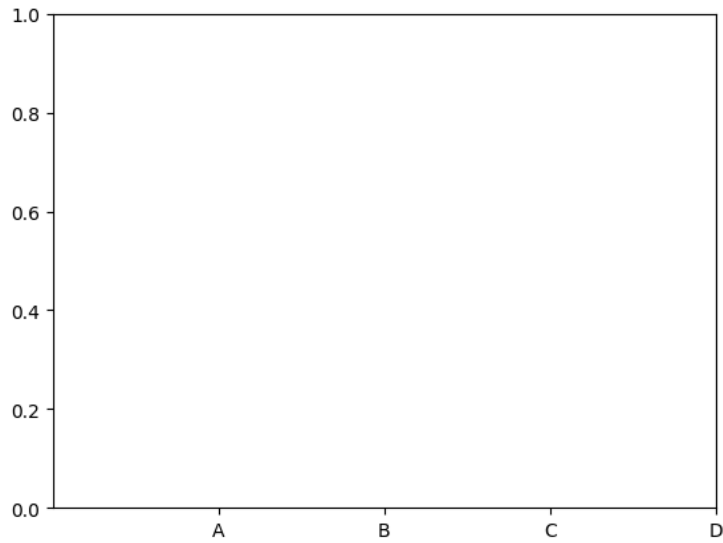
```
plt.xticks([1,2,3,4])
plt.yticks([10,20,30,40])
```

```
([<matplotlib.axis.YTick at 0x79e05842ab70>,
 <matplotlib.axis.YTick at 0x79e05842a0c0>,
 <matplotlib.axis.YTick at 0x79e0459d9a30>,
 <matplotlib.axis.YTick at 0x79e0459f8230>],
 [Text(0, 10, '10'), Text(0, 20, '20'), Text(0, 30, '30'), Text(0, 40, '40')])
```



```
plt.xticks([1,2,3,4], ["A", "B", "C", "D"])
```

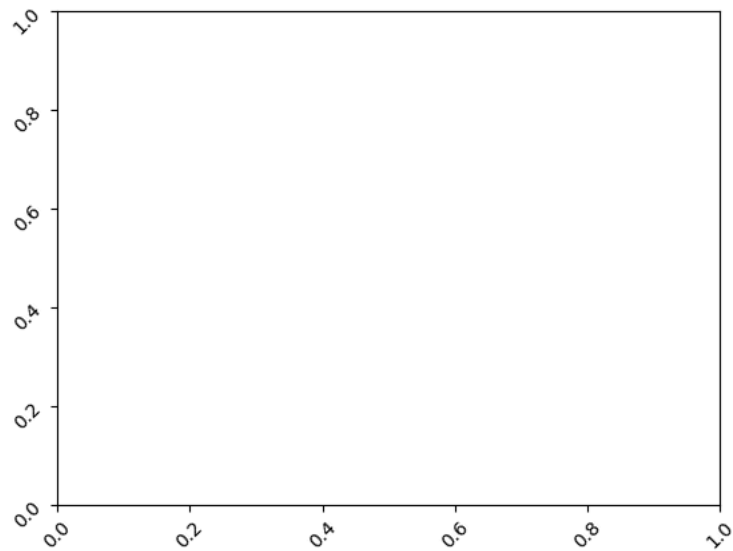
```
([<matplotlib.axis.XTick at 0x79e0459fa450>,
<matplotlib.axis.XTick at 0x79e059118560>,
<matplotlib.axis.XTick at 0x79e0456e1e50>,
<matplotlib.axis.XTick at 0x79e0456e2840>],
[Text(1, 0, 'A'), Text(2, 0, 'B'), Text(3, 0, 'C'), Text(4, 0, 'D')])
```



Rotating Tick Labels

```
plt.xticks(rotation=45)
plt.yticks(rotation=45)
```

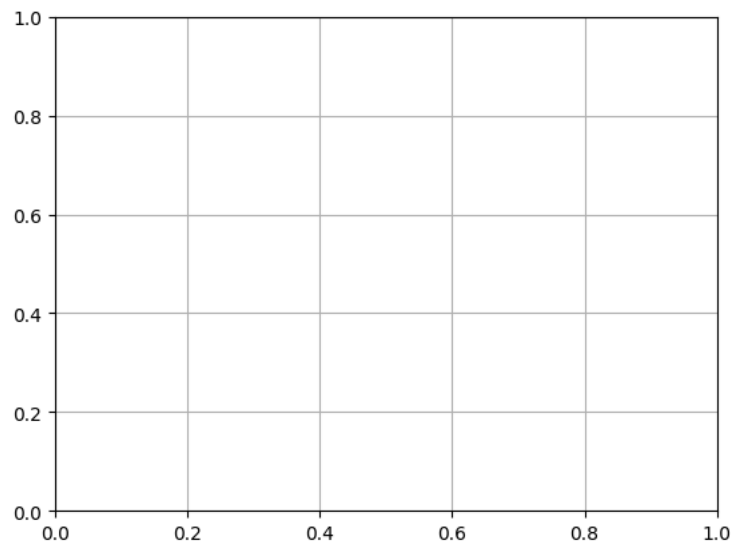
```
(array([0. , 0.2, 0.4, 0.6, 0.8, 1. ]),
[Text(0, 0.0, '0.0'),
Text(0, 0.2, '0.2'),
Text(0, 0.4, '0.4'),
Text(0, 0.6000000000000001, '0.6'),
Text(0, 0.8, '0.8'),
Text(0, 1.0, '1.0')])
```



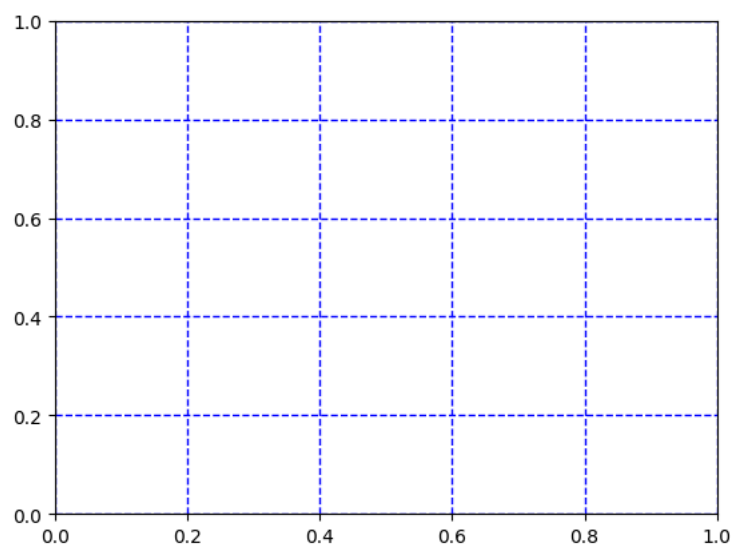
Grid Control

Grids improve readability

```
plt.grid(True)
```

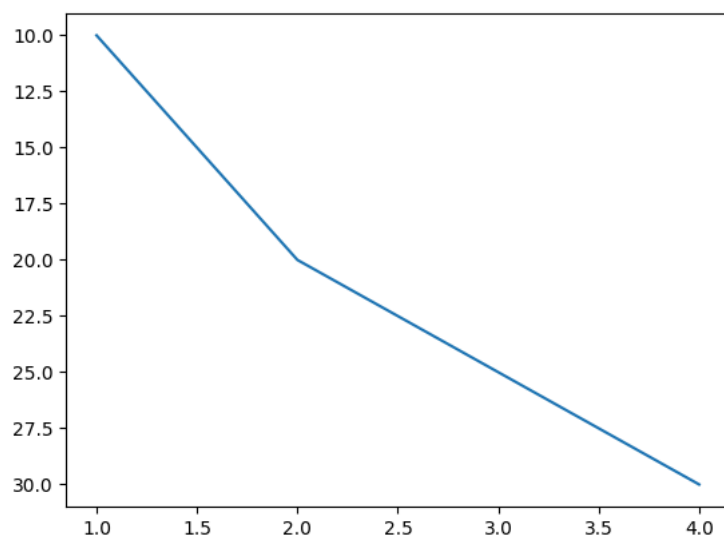


```
plt.grid(color='blue', linestyle='--', linewidth=1)
```



Inverting Axes

```
plt.plot([1,2,3,4], [10,20,25,30])  
plt.gca().invert_yaxis()
```

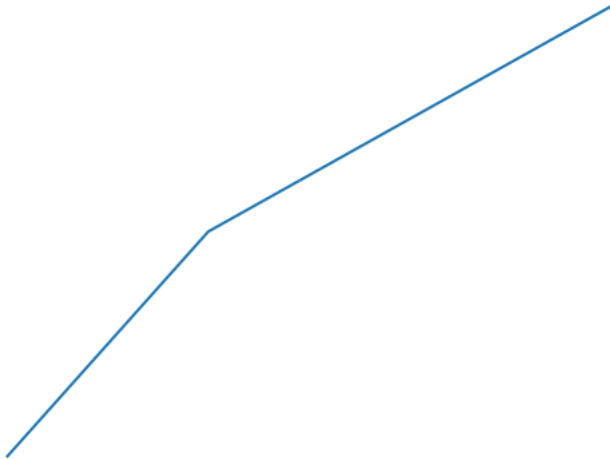


Multiple Axes (Twin Axes)

Removing Axes

```
plt.plot([1,2,3,4], [10,20,25,30])  
plt.axis('off')
```

```
(np.float64(0.85), np.float64(4.15), np.float64(9.0), np.float64(31.0))
```

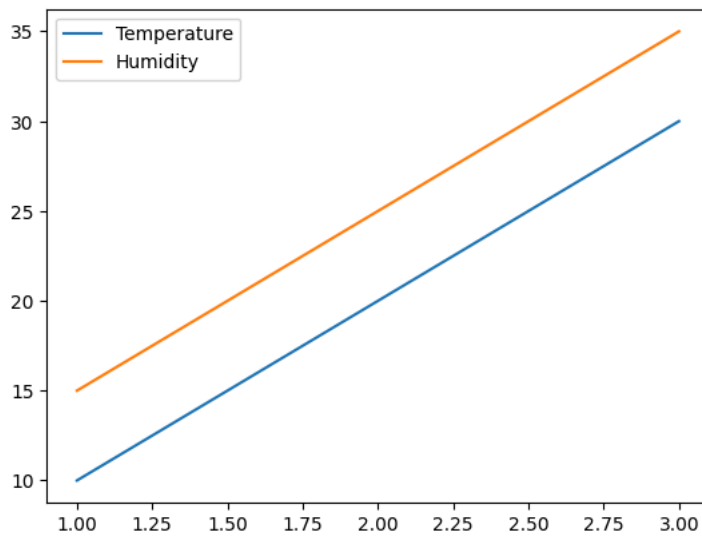


Legend

Used when multiple data lines are plotted.

```
x = [1,2,3]  
y = [10,20,30]  
plt.plot(x, y, label="Temperature")  
plt.plot(x, [15,25,35], label="Humidity")  
  
plt.legend()
```

```
<matplotlib.legend.Legend at 0x79e045613e00>
```



Annotations

Annotations help highlight specific points or add explanations.

```
plt.plot(x, y)  
  
plt.annotate("Peak Value",  
            xy=(3,30),      # point to highlight  
            xytext=(2,25),   # text position  
            arrowprops=dict(facecolor='blue'))  
  
plt.show()
```

